

Введение

Начало работы

Базовая настройка

Основные функции

Умная подсказка

Конфигурация проекта

Расширенные рабочие
процессы

Автоматизация и хуки

Расширения MCP

Отладка и тестирование

Полезные советы

Основные правила

Распространенные ошибки

Более 50 советов и рекомендаций

Конечная Цель Код Клода Руководство

Как старшие инженеры Anthropic используют Claude Code, чтобы выпускать продукты в пять раз быстрее

Claude Code невероятно функционален при правильной настройке, но большинство разработчиков используют его возможности лишь частично. Это подробное руководство содержит более 50 практических советов, которые помогут вам по-новому взглянуть на использование Claude Code — от основ для начинающих до продвинутой автоматизации, которая повысит вашу продуктивность в 10 раз.

Приступая к работе

1

Правильная установка Claude Code

Редактировать с помощью  Claude 

У вас есть три варианта, с чего начать:

Локальная установка: Скачайте с сайта Anthropic и установите на свой компьютер

Удаленный сервер: Установите на AWS, Digital Ocean или любой другой облачный сервер, чтобы писать код из любого места

В других инструментах: Используйте Claude Code в Cursor, Windsurf или VS Code

Профессиональный совет: Удаленная установка позволяет писать код с телефона с помощью таких приложений, как Termius!

2 Выберите тарифный план с умом

Pro-план (\$20 в месяц): подходит для обучения и небольших проектов

Max-план (\$100 в месяц): лучший вариант для серьезных разработчиков — в 5 раз больше лимитов

Max 20X (\$200 в месяц): для активных пользователей, которые программируют целыми днями

Откажитесь от использования API, если только ваша компания не платит за него — это слишком дорого.

3 Освойте три режима ввода

Нажмите Shift + Tab для переключения режимов:

Режим редактирования: запрашивает разрешение перед внесением изменений (по умолчанию)

Режим автоматического принятия: вносит изменения без запроса (оптимально для большинства задач)

Режим планирования: создает планы без написания кода (идеально для обдумывания проблем)

Редактировать с помощью  Claude

Основные настройки и команды

4 Настройка многострочных подсказок

Запустите `/terminal-setup` один раз, чтобы включить `Shift + Enter` многострочные подсказки. Это необходимо для выполнения сложных инструкций.

5 Подключите вашу IDE

Используйте `/ide` для подключения VS Code, Cursor или JetBrains. Это даст Claude доступ к выбранному вами коду и диагностике IDE.

6 Основные сочетания клавиш

`CMD/CTRL + Escape` : Быстрое открытие Claude Code

`CMD/CTRL + L` : Очистить экран

`ESC + ESC` : Перейти к предыдущему запросу

`CMD/CTRL + R` : Подробный вывод

`Shift + Enter` : Новая строка (после настройки терминала)

7 Используйте функцию возобновления

Если Claude Code зависнет или вы случайно закроете его, используйте `/resume` для возобновления работы с того места, на котором остановились. Это работает даже после отключения электричества!

8 Как профессионально работать с длинными запросами

Для сложных подсказок:

1. Нажмите `CMD/CTRL + N` для открытия нового буфера
2. Введите всю команду с форматированием
3. Выделите все (`CMD/CTRL + A`) и скопируйте
4. Вставить в Claude Code

Таким образом, все сводится к одной понятной строке.

Основные характеристики

9 Используйте списки дел для решения сложных задач

Главная фишка Claude Code — автоматические списки дел. Для крупных проектов перед написанием кода явно попросите Claude «сначала создать список дел». Это позволяет избежать заикливания и поддерживать порядок в работе.

10 Режим Master Bash (Мастер - удар)

Claude может выполнять за вас команды терминала. Попросите его:

Чтение файлов в папках для получения контекста

Обработка команд `git`

Установка пакетов

Запуск тестов

Это значит, что вам редко придется покидать Claude Code.

11 **Работайте непосредственно с изображениями**

Перетаскивайте скриншоты или копируйте изображения в Claude Code. Идеально подходит для:

Запросы «Сделайте так, чтобы это выглядело вот так»

Устранение проблем с пользовательским интерфейсом

Публикация макетов

12 **Отслеживайте Свои Расходы**

Запустите `prx csusage`, чтобы просмотреть подробную информацию об использовании и стоимости токенов. Используйте `blocks --live` для мониторинга в режиме реального времени.

13 **Не бойтесь перебивать**

Нажмите `Escape` один раз, чтобы остановить Клода, и дважды, чтобы вернуться к предыдущему запросу. Лучше перенаправить его раньше, чем тратить токены впустую.

Умное Подсказывание

14 **Управляйте силой мышления Клода**

Используйте эти ключевые слова, чтобы контролировать мыслительный процесс Клода:

"think about this..." - Базовое мышление (быстрое)

Редактировать с помощью  Claude

"think harder about..." - Более глубокий анализ (сложные проблемы)

"ultrathink about..." - Максимальное мышление (критически важные решения)

15 **Используйте субагентов для решения масштабных задач**

Попросите Клода «использовать субагентов для рефакторинга этой кодовой базы». Клод сделает следующее:

Разделите работу на части

Создайте несколько агентов для параллельной работы

Автоматически координируйте все процессы

Объединяйте результаты

16 **Выполнение задач в циклах**

Для итеративного исправления скажите: «Запускайте сборку в цикле и исправляйте все ошибки по мере их появления». Claude будет продолжать работу и вносить исправления, пока все не заработает.

17 **Режим планирования с использованием заемных средств**

Прежде чем приступить к написанию кода, используйте режим планирования, чтобы:

Продумайте архитектурные решения

Спланируйте сложные функции

Отладьте сложные проблемы

Дважды нажмите Shift + Tab для перехода в режим просмотра только плана.

18 Используйте очередь сообщений

Вам не нужно ждать, пока Claude закончит работу. Вводите новые сообщения, пока он работает — они будут добавлены в очередь и обработаны по порядку.

Конфигурация проекта

19 Создание мощного файла `claude.md`

Это память вашего проекта. Claude считывает этот файл при каждом запуске. Включить:

Стандарты и предпочтения в кодировании

Обзор архитектуры проекта

Правила рабочего процесса Git

Требования к тестированию

Стандарты документации

20 Автоматическая генерация `claude.md`

Не пишите это вручную! Попросите Claude: «Проанализируй мой проект и создай полный файл `claude.md` со всеми обнаруженными правилами и стандартами».

21 Использование вложенных файлов `claude.md`

Создание правил для отдельных каталогов:

текст

```
project/  
├─ claude.md (global rules)  
└─ frontend/
```

Редактировать с помощью  Claude

```
| └─ claude.md (frontend rules)
└─ backend/
    └─ claude.md (backend rules)
```

22 Инициализация существующих проектов

Для существующих кодовых баз выполните команду `/init` для автоматической генерации:

Обнаруженные соглашения

Документация по структуре файлов

Анализ зависимостей

Шаблоны кода

23 Добавляйте Правила на лету

Введите `# "Always use async/await instead of .then()"` в любом месте диалога. Claude автоматически добавит его в ваш файл `claude.md`.

Основные правила для каждого проекта

Правило контроля версий

уценка

Git Workflow

- Create feature branches for all changes
- Commit frequently with descriptive messages
- Never push directly to main branch
- Add and commit automatically when tasks complete

Правило качества кода

Редактировать с помощью  Claude

уценка

Code Quality (CRITICAL)

ALWAYS run IDE diagnostics after editing files

Fix all linting and type errors before completing tasks

This step must NEVER be skipped

Documentation Rule

markdown

Documentation

- Update README.md when adding new features
- Create inline comments for complex logic
- Generate API docs for new endpoints
- Keep change logs updated

Testing Rule

markdown

Testing Requirements

- Write tests for all new features
- Run existing tests before completing tasks
- Focus on end-to-end tests over unit tests
- Use test-driven development for complex features

Final Thoughts

Claude Code is incredibly powerful when configured correctly. The key is to:

Set up proper automation with hooks and rules

Use the right prompting techniques for your tasks

Leverage MCP extensions for enhanced capabilities

Think like a product manager rather than a code reviewer

Редактировать с помощью  Claude

Customize everything to match your workflow

Start with the basics, gradually add automation, and soon you'll be shipping features faster than ever before.

REMEMBER

Claude Code is not just a coding assistant - it's a complete development environment that can handle research, documentation, testing, and deployment when properly configured. The developers who master these techniques will have a massive advantage in the AI-powered future of software development.